



S.E.P. TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO de Tuxtepec

Prácticas Python

ASIGNATURA:
“Ciencia de Datos”

PRESENTA:

Dimas Montoro Héctor Iván 22350317

CARRERA:
**INGENIERÍA EN SISTEMAS
COMPUTACIONALES**

DOCENTE:
Julio Aguilar Carmona

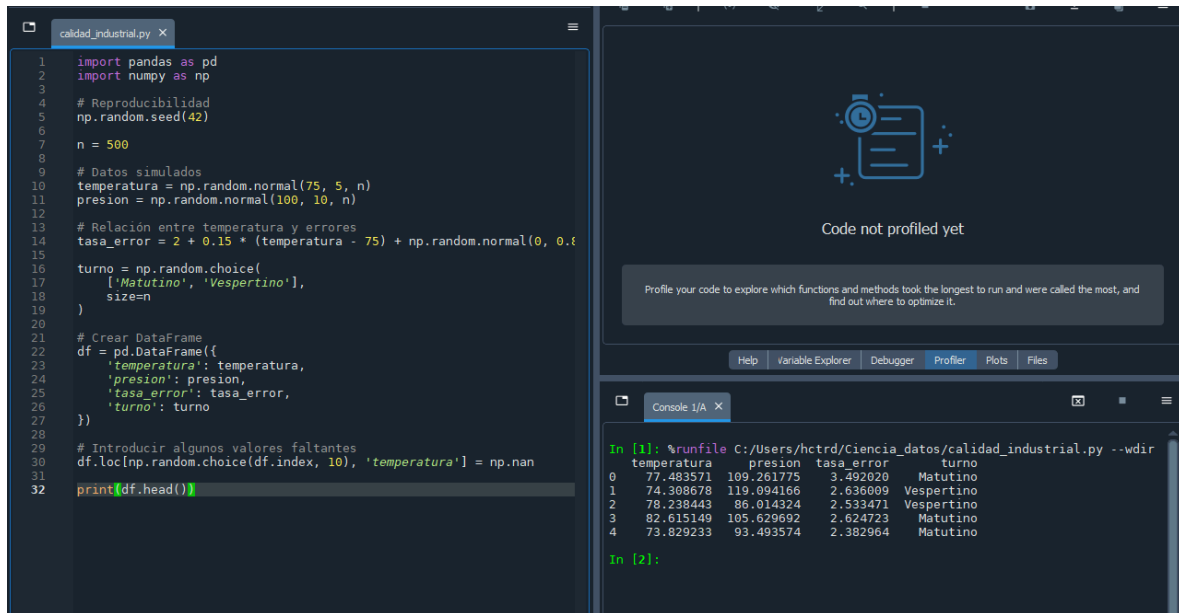
Mayo 2026

Índice

Índice	1
Práctica 2: Análisis de Calidad Industrial (Industria 4.0)	2
Fase A: Análisis Exploratorio.	2
Fase B: Agrupación y Segmentación.	3
Fase C: Visualización e Interpretación.	4
Conclusión	6
Práctica 3: Análisis de Eficiencia en Logística Global.	7
Fase 1: Calidad de Datos.	7
Fase 2: Análisis de Relaciones.	8
Fase 3: Comparativa de Modalidades	8
Fase 4: Visualización.	9
Conclusión	11
Práctica 4: Reducción de Dimensionalidad en Monitoreo Industrial	12
Fase 1: Preparación	13
Fase 2: Ejecución de PCA	14
Fase 3: Selección de Componentes	15
Fase 4: Visualización	17
Conclusión	18
Práctica 5: Reducción de Dimensiones en Tráfico de Red (Cyber-Systems)	19
Paso 1: Generación y Exploración de Datos	19
Paso 2: Estandarización (Scaling)	20
Paso 3: Análisis de Varianza (Varianza Explicada)	22
Paso 4: Interpretación de Cargas (Loadings)	22
Paso 5: Visualización (Biplot)	23
Conclusión	24

Práctica 2: Análisis de Calidad Industrial (Industria 4.0)

Como primer paso creamos un DataFrame con datos generados gracias a `numpy.random`.



```
1 import pandas as pd
2 import numpy as np
3
4 # Reproducibilidad
5 np.random.seed(42)
6
7 n = 500
8
9 # Datos simulados
10 temperatura = np.random.normal(75, 5, n)
11 presion = np.random.normal(100, 10, n)
12
13 # Relación entre temperatura y errores
14 tasa_error = 2 + 0.15 * (temperatura - 75) + np.random.normal(0, 0.5, n)
15
16 turno = np.random.choice(
17     ['Matutino', 'Vespertino'],
18     size=n
19 )
20
21 # Crear DataFrame
22 df = pd.DataFrame({
23     'temperatura': temperatura,
24     'presion': presion,
25     'tasa_error': tasa_error,
26     'turno': turno
27 })
28
29 # Introducir algunos valores faltantes
30 df.loc[np.random.choice(df.index, 10), 'temperatura'] = np.nan
31
32 print(df.head())
```

Code not profiled yet

Profile your code to explore which functions and methods took the longest to run and were called the most, and find out where to optimize it.

Help Variable Explorer Debugger Profiler Plots Files

Console 1/A

```
In [1]: %runfile C:/Users/hctrd/Ciencia_datos/calidad_industrial.py --wdir
temperatura presion tasa_error turno
0 77.483571 109.261775 3.492020 Matutino
1 74.308678 119.094166 2.636009 Vespertino
2 78.238443 86.014324 2.533471 Vespertino
3 82.615149 105.629692 2.624723 Matutino
4 73.829233 93.493574 2.382964 Matutino

In [2]:
```

Fase A: Análisis Exploratorio.

Posteriormente calculamos media, moda y desviación estándar usando `.mean`, `.median` y `.std` respectivamente.

```
31 media = df['temperatura'].mean()
32 mediana = df['temperatura'].median()
33 desviacion = df['temperatura'].std()
34
35 print("Media:", media)
36 print("Mediana:", mediana)
37 print("Desviación estándar:", desviacion)
```

```
0 77.483571 109.261775 3.492020 Matutino
1 74.308678 119.094166 2.636009 Vespertino
2 78.238443 86.014324 2.533471 Vespertino
3 82.615149 105.629692 2.624723 Matutino
4 73.829233 93.493574 2.382964 Matutino

In [2]: %runfile C:/Users/hctrd/Ciencia_datos/calidad_industrial.py --wdir
Media: 75.03785373480814
Mediana: 75.08501433377666
Desviación estándar: 4.833542146197055
```

Ahora buscamos valores faltantes filtrando el DataFrame con `df.isnull()` lo que nos devuelve la cantidad de campos que se encuentran vacíos y los complementamos usando la media calculada anteriormente.

```
36 print(df.isnull().sum())

In [2]: %runfile C:/Users/hctrd/Ciencia_datos/calidad_industrial.py --wdir
Media: 75.03785373480814
Mediana: 75.08501433377666
Desviación estándar: 4.833542146197055

In [3]: %runfile C:/Users/hctrd/Ciencia_datos/calidad_industrial.py --wdir
temperatura    10
presion         0
tasa_error      0
turno           0
dtype: int64
```

Para determinar la correlación de Pearson entre la temperatura y la tasa de errores usando `.corr()` lo que nos devuelve un valor entre 1 y -1 siendo 1=Relación Positiva y -1=Relación Negativa.

```
44
45 print("Correlación:", correlacion)

dtype: int64
In [12]: %runfile C:/Users/hctrd/Ciencia_datos/calidad_industrial.py --wdir
Correlación: 0.6232913862589353
```

Fase B: Agrupación y Segmentación.

Para esta fase lo primero que haremos es una comparativa entre la tasa de errores entre ambos turnos (Matutino y Vespertino) usando `.groupby` para filtrar solo la columna de turnos y calculando la media de cada uno.

```
method='pearson'
)
promedio_turno = df.groupby('turno')['tasa_error'].mean()
print(promedio_turno)

In [13]: %runfile C:/Users/hctrd/Ciencia_datos/calidad_industrial.py --wdir
turno
Matutino    2.062697
Vespertino   2.118458
Name: tasa_error, dtype: float64

In [14]:
```

Si queremos que el programa interprete automáticamente la relación y avise si hay una diferencia significativa entre ambos turnos haremos una prueba de t-Student y agregaremos una condición de 0.05 para juzgar la diferencia.

```
from scipy.stats import ttest_ind

matutino = df[df['turno'] == 'Matutino']['temperatura']
vespertino = df[df['turno'] == 'Vespertino']['temperatura']

t_stat, p_valor = ttest_ind(
    matutino,
    vespertino
)

print("Estadístico t:", t_stat)
print("Valor p:", p_valor)

if p_valor < 0.05:
    print("Existe diferencia significativa.")
else:
    print("No existe diferencia significativa.")
```

```
In [14]: %runfile C:/Users/hctrd/Ciencia_datos/calidad_industrial.py --wdir
Estadístico t: -0.5993971077281167
Valor p: 0.5491806943042543
No existe diferencia significativa.

In [15]:
```

Fase C: Visualización e Interpretación.

Para las gráficas de caja y dispersión usaremos pyplot de la librería matplotlib y seaborn.

```
calidad_industrial.py X
'temperatura': temperatura,
'presion': presion,
'tasa_error': tasa_error,
'turno': turno
})

# Introducir algunos valores faltantes
df.loc[np.random.choice(df.index, 10), 'temperatura'] = np.nan

media = df['temperatura'].mean()
mediana = df['temperatura'].median()
desviacion = df['temperatura'].std()

df['temperatura'] = df['temperatura'].fillna(
    media
)

correlacion = df['temperatura'].corr(
    df['tasa_error'],
    method='pearson'
)

promedio_turno = df.groupby('turno')['tasa_error'].mean()

from scipy.stats import ttest_ind

matutino = df[df['turno'] == 'Matutino']['temperatura']
vespertino = df[df['turno'] == 'Vespertino']['temperatura']

t_stat, p_valor = ttest_ind(
    matutino,
    vespertino
)

import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(8,5))

sns.boxplot(
    x='turno',
    y='tasa_error',
    data=df
)

plt.title('Distribución de errores por turno')
plt.show()

plt.figure(figsize=(8,5))

sns.regplot(
    x='temperatura',
    y='tasa_error',
    data=df
)

plt.title('Temperatura vs Tasa de Error')
plt.show()
```

Distribución de errores por turno

Help Variable Explorer Debugger Profiler Plots Files

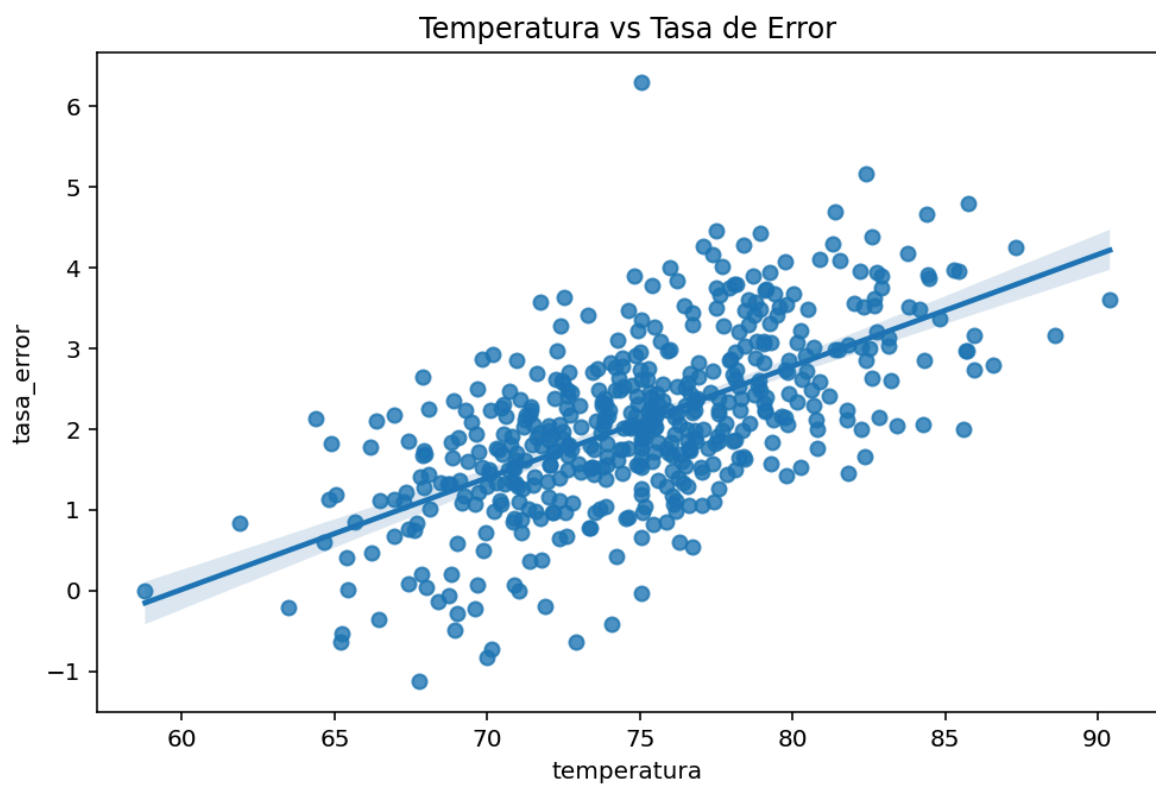
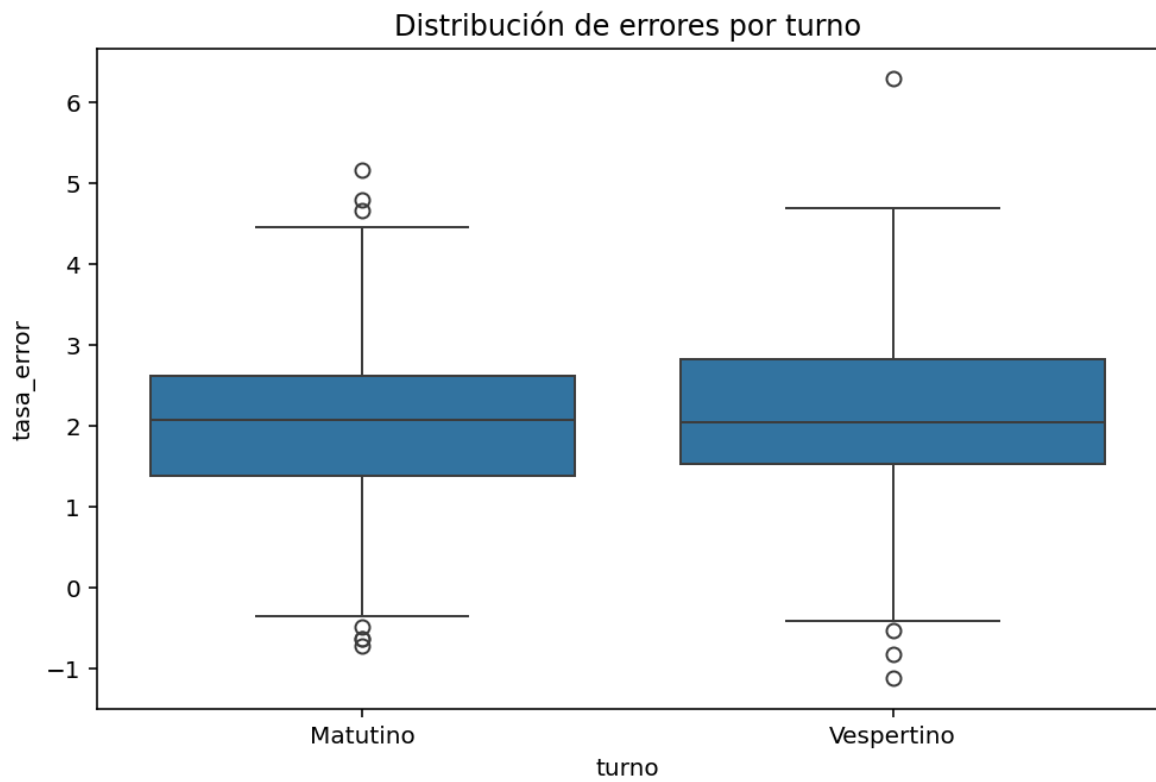
```
In [14]: %runfile C:/Users/hctrd/Ciencia_datos/calidad_industrial.py --wdir
Estadístico t: -0.5993971077281167
Valor p: 0.5491806943042543
No existe diferencia significativa.

In [15]: %runfile C:/Users/hctrd/Ciencia_datos/calidad_industrial.py --wdir
```

Important

Figures are displayed in the Plots pane by default. To make them also appear inline in the console, you need to uncheck "Mute inline plotting" under the options menu of Plots.

```
In [16]: %runfile C:/Users/hctrd/Ciencia_datos/calidad_industrial.py --wdir
In [17]: %runfile C:/Users/hctrd/Ciencia_datos/calidad_industrial.py --wdir
In [18]:
```

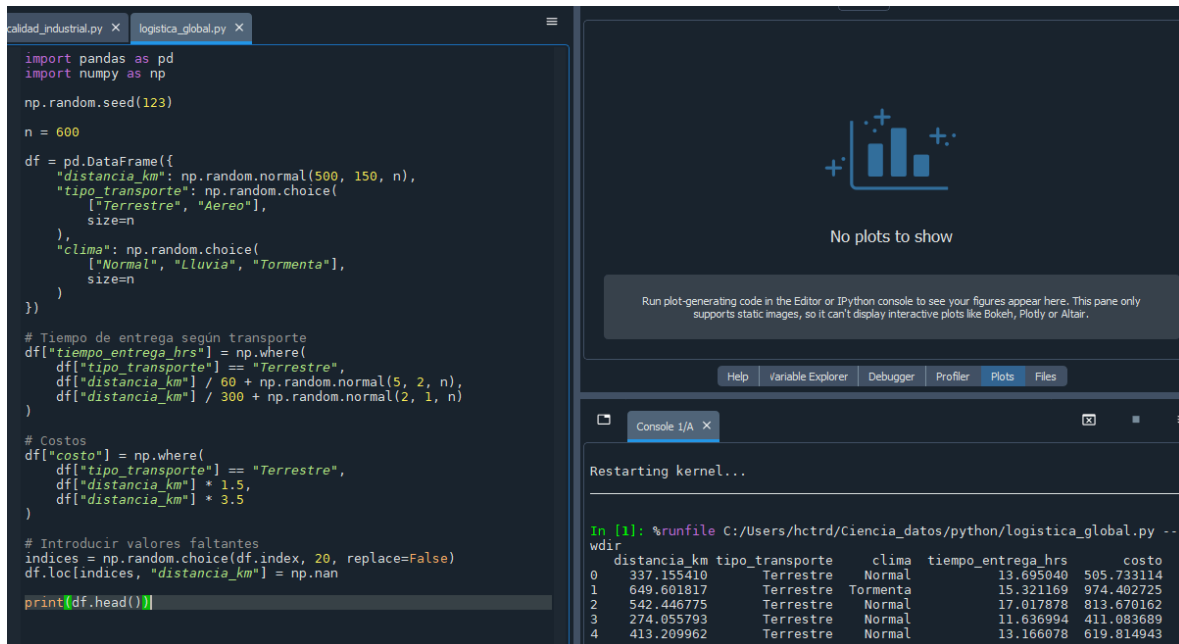


Conclusión

El análisis estadístico mostró una correlación positiva entre la temperatura de operación y la tasa de error, lo que indica que incrementos en la temperatura están asociados con una mayor cantidad de defectos en la producción. El gráfico de dispersión confirmó esta tendencia mediante una línea de regresión ascendente. Asimismo, el boxplot permitió identificar algunos valores atípicos en la tasa de error. Si la correlación observada es moderada o fuerte, la empresa debería considerar invertir en mejores sistemas de enfriamiento y monitoreo térmico, ya que un mejor control de la temperatura podría reducir la cantidad de productos defectuosos y mejorar la calidad general del proceso de manufactura.

Práctica 3: Análisis de Eficiencia en Logística Global.

De igual forma comenzamos creando un DataFrame.



The screenshot shows a Jupyter Notebook with two tabs: 'calidad_industrial.py' and 'logistica_global.py'. The code in 'logistica_global.py' creates a DataFrame with columns: 'distancia_km', 'tipo_transporte', 'clima', 'tiempo_entrega_hrs', and 'costo'. It uses random values for 'distancia_km' and 'clima', and calculated values for 'tiempo_entrega_hrs' and 'costo' based on 'distancia_km' and 'tipo_transporte'. The console output shows the first five rows of the DataFrame.

```
import pandas as pd
import numpy as np

np.random.seed(123)

n = 600

df = pd.DataFrame({
    "distancia_km": np.random.normal(500, 150, n),
    "tipo_transporte": np.random.choice(
        ["Terrestre", "Aereo"],
        size=n
    ),
    "clima": np.random.choice(
        ["Normal", "Lluvia", "Tormenta"],
        size=n
    )
})

# Tiempo de entrega según transporte
df["tiempo_entrega_hrs"] = np.where(
    df["tipo_transporte"] == "Terrestre",
    df["distancia_km"] / 60 + np.random.normal(5, 2, n),
    df["distancia_km"] / 300 + np.random.normal(2, 1, n)
)

# Costos
df["costo"] = np.where(
    df["tipo_transporte"] == "Terrestre",
    df["distancia_km"] * 1.5,
    df["distancia_km"] * 3.5
)

# Introducir valores faltantes
indices = np.random.choice(df.index, 20, replace=False)
df.loc[indices, "distancia_km"] = np.nan

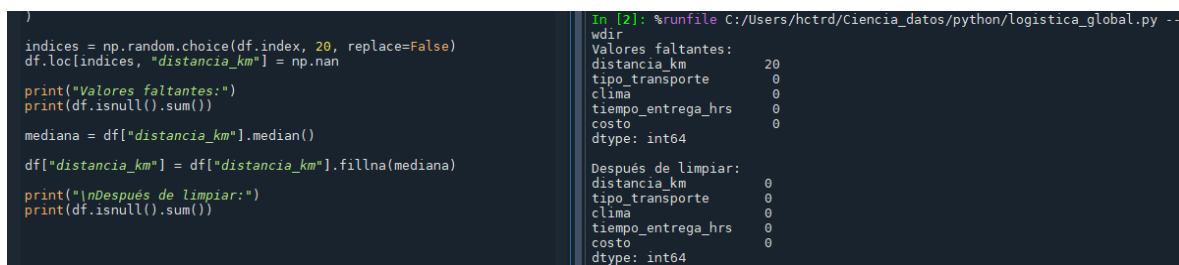
print(df.head())
```

Restarting kernel...

```
In [1]: %runfile C:/Users/hctrd/Ciencia_datos/python/logistica_global.py --
wdir
distancia_km  tipo_transporte  clima  tiempo_entrega_hrs  costo
0    337.155410      Terrestre   Normal      13.695040    505.733114
1    649.601817      Terrestre  Tormenta      15.321169    974.402725
2    542.446775      Terrestre   Normal      17.017878    813.670162
3    274.055793      Terrestre   Normal      11.636994    411.083689
4    413.209962      Terrestre   Normal      13.166078    619.814943
```

Fase 1: Calidad de Datos.

Complementamos los valores nulos usando la mediana, como paso extra imprimimos los datos antes y después de ser modificados.



The screenshot shows the continuation of the Jupyter Notebook. The code replaces null values in the 'distancia_km' column with the median value. The console output shows the number of null values before and after the replacement, and the data types of the columns.

```
indices = np.random.choice(df.index, 20, replace=False)
df.loc[indices, "distancia_km"] = np.nan

print("Valores faltantes:")
print(df.isnull().sum())

mediana = df["distancia_km"].median()

df["distancia_km"] = df["distancia_km"].fillna(mediana)

print("\nDespués de limpiar:")
print(df.isnull().sum())
```

```
In [2]: %runfile C:/Users/hctrd/Ciencia_datos/python/logistica_global.py --
wdir
Valores faltantes:
distancia_km      20
tipo_transporte    0
clima              0
tiempo_entrega_hrs 0
costo              0
dtype: int64

Después de limpiar:
distancia_km      0
tipo_transporte    0
clima              0
tiempo_entrega_hrs 0
costo              0
dtype: int64
```


Calculamos la media y desviación estándar del tiempo de entregas y las redondeamos conservando 2 dígitos decimales.

```
df["distancia_km"] = df["distancia_km"].fillna(media)
media = df["tiempo_entrega_hrs"].mean()
desviacion = df["tiempo_entrega_hrs"].std()
print("Media:", round(media, 2))
print("Desviación estándar:", round(desviacion, 2))
```

```
In [3]: %runfile C:/Users/hctrd/Ciencia_datos/python/logistica_global.py --
wdir
Media: 8.39
Desviación estándar: 5.31
In [4]:
```

Fase 2: Análisis de Relaciones.

Se calculó el coeficiente de Pearson entre la distancia y el tiempo de entregas para detectar si hay una fuerte relación entre ambas tomando como referencia un valor entre 1 y 0, siendo 1 una relación fuerte y 0 una relación débil.

```
correlacion = df["distancia_km"].corr(
    df["tiempo_entrega_hrs"],
    method="pearson"
)
print("Correlación:", round(correlacion, 4))
```

```
In [4]: %runfile C:/Users/hctrd/Ciencia_datos/python/logistica_global.py --
wdir
Correlación: 0.2585
In [5]:
```

Fase 3: Comparativa de Modalidades

Para comparar los tiempos de entrega y los costos de los tipos de transportes con los que cuenta la empresa se agruparon y promediaron ambos datos.

```
df["tiempo_entrega_hrs"],
method="pearson"
)
resumen = df.groupby("tipo_transporte").agg({
    "tiempo_entrega_hrs": "mean",
    "costo": "mean"
})
print(resumen)
```

```
In [5]: %runfile C:/Users/hctrd/Ciencia_datos/python/logistica_global.py --
wdir

```

tipo_transporte	tiempo_entrega_hrs	costo
Aereo	3.684571	1735.723143
Terrestre	13.287028	749.863141

```
In [6]:
```

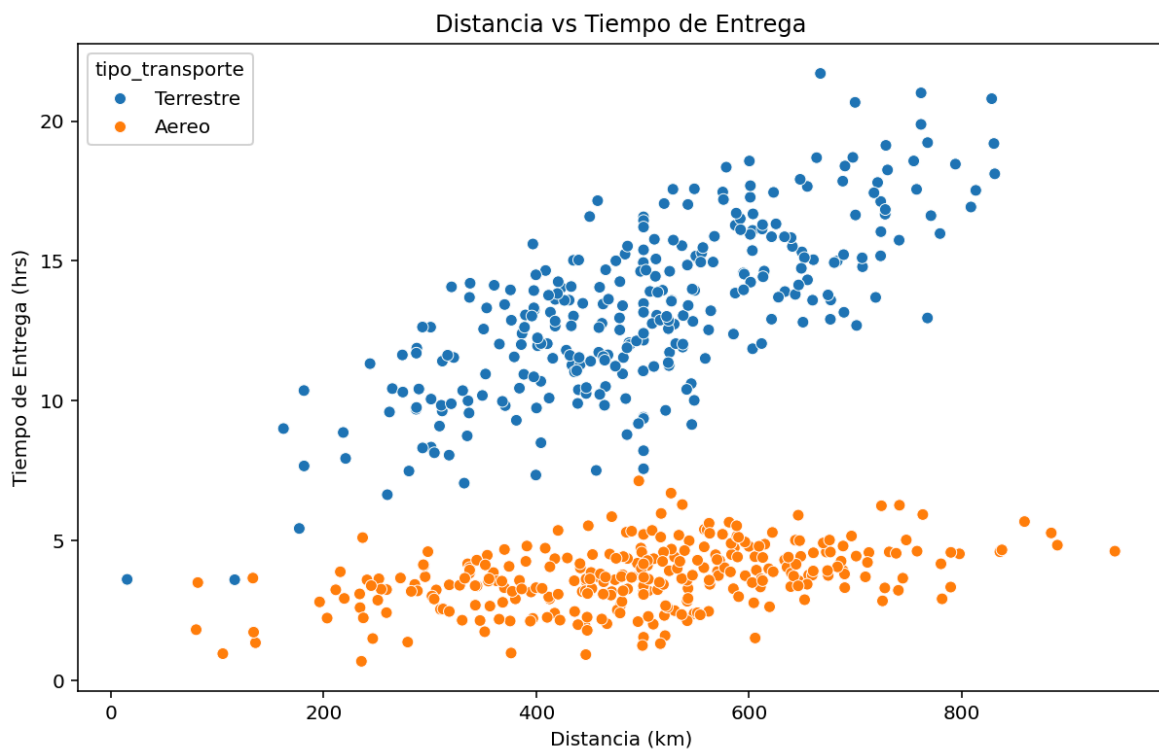
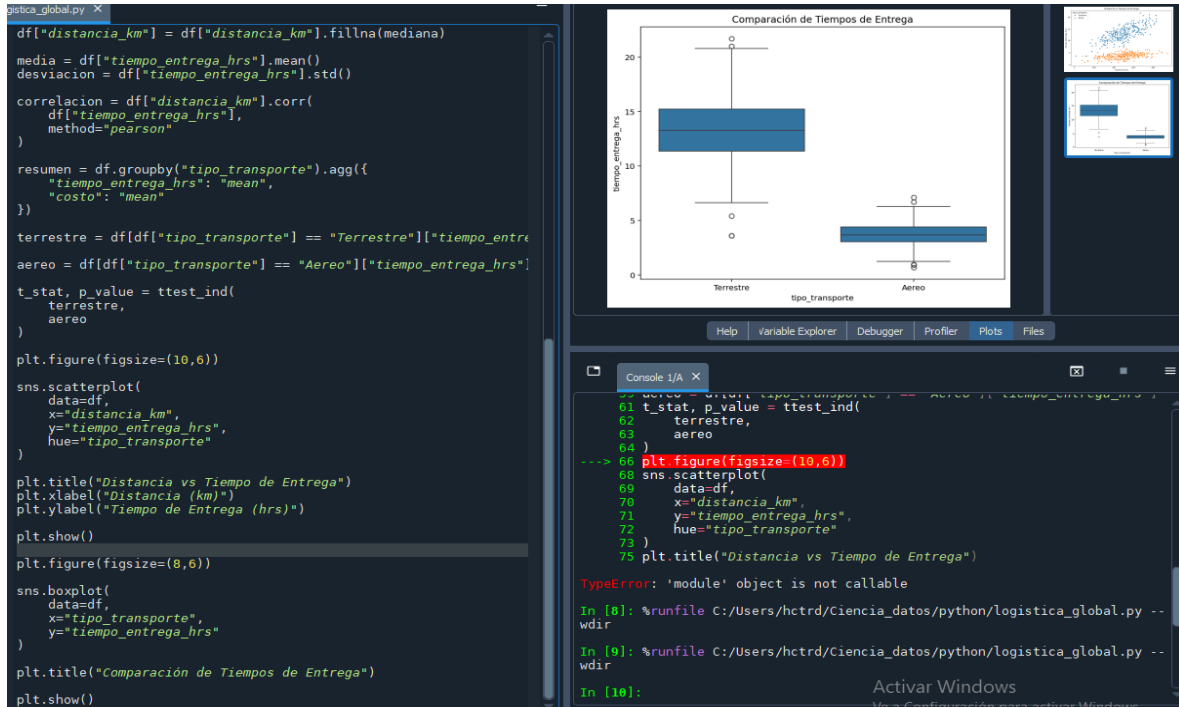
Se usó una prueba de t-Student para identificar si existe una diferencia significativa entre el tiempo de entrega.

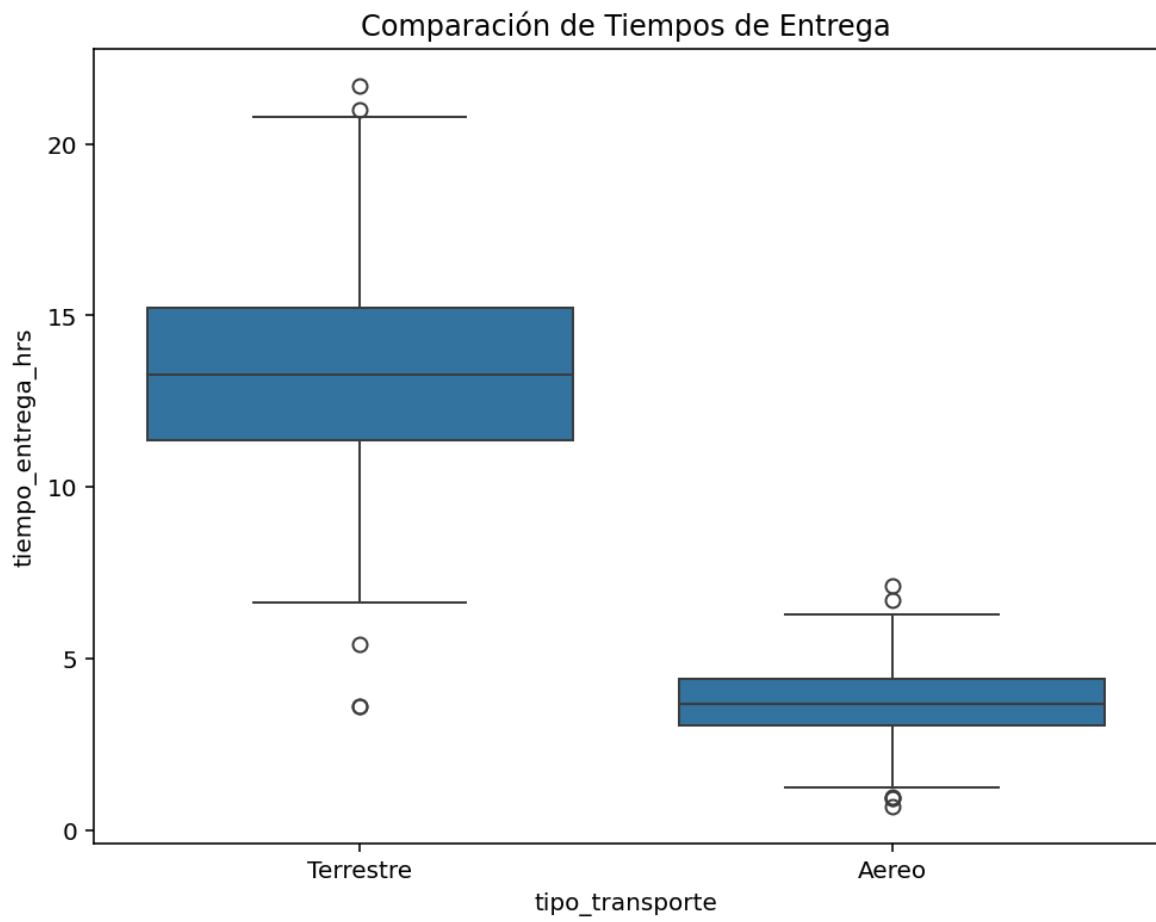
```
terrestre = df[df["tipo_transporte"] == "Terrestre"]["tiempo_entrega_hrs"]
aereo = df[df["tipo_transporte"] == "Aereo"]["tiempo_entrega_hrs"]
t_stat, p_value = ttest_ind(
    terrestre,
    aereo
)
print("t =", round(t_stat, 4))
print("p-value =", p_value)
```

```
In [6]: %runfile C:/Users/hctrd/Ciencia_datos/python/logistica_global.py --
wdir
t = 52.0692
p-value = 2.472129041422765e-224
In [7]:
```

Fase 4: Visualización.

Para graficar la relación entre la distancia y tiempo de las distintas modalidades de transporte se usó Scatterplot y Boxplot.



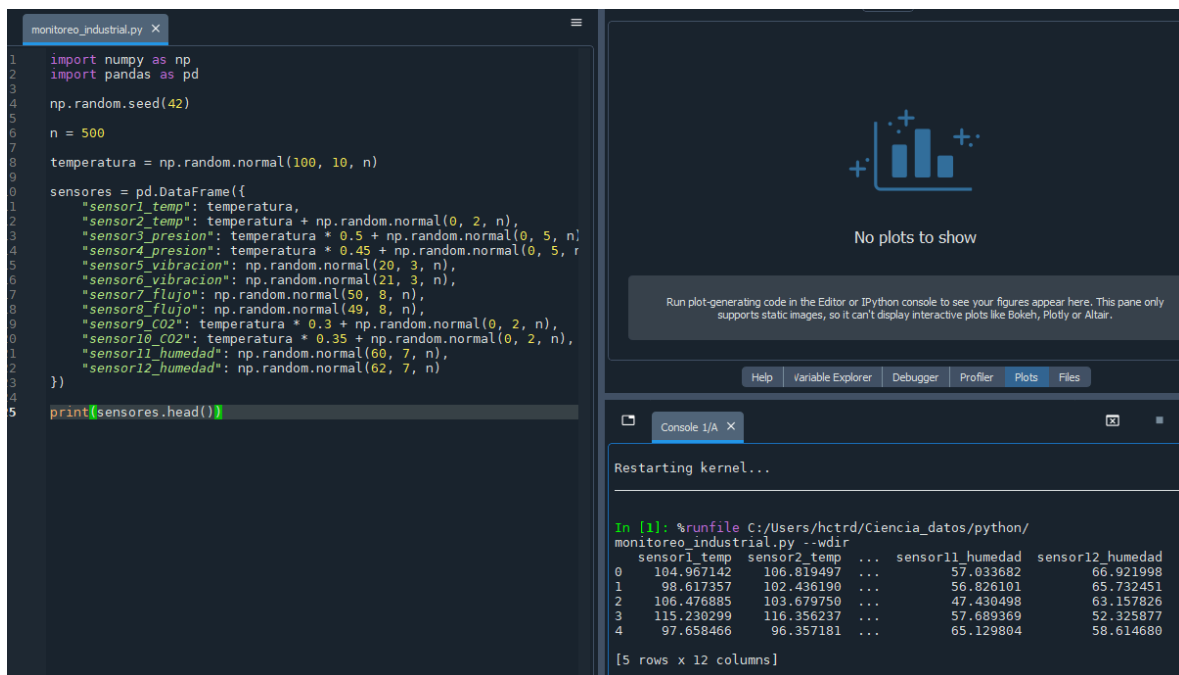


Conclusión

Existe evidencia estadística de que el tipo de transporte afecta significativamente el tiempo de entrega. El transporte aéreo puede ser una alternativa para mejorar la satisfacción del cliente, aunque implica mayores costos.

Práctica 4: Reducción de Dimensionalidad en Monitoreo Industrial

El Data Frame sintético que se generará contará con 12 columnas simulando datos de 12 sensores distintos.



The screenshot displays a Jupyter Notebook environment. The left pane shows a Python script in a file named `monitoreo_industrial.py`. The script imports `numpy` and `pandas`, sets a random seed, and generates a synthetic dataset with 500 rows and 12 columns. The columns are named `sensor1_temp` through `sensor12_humedad`. The right pane shows the output of the script, which is a `DataFrame` object. Below the output, there is a console window showing the execution of the script and the resulting `DataFrame` structure.

```
monitoreo_industrial.py X
1 import numpy as np
2 import pandas as pd
3
4 np.random.seed(42)
5
6 n = 500
7
8 temperatura = np.random.normal(100, 10, n)
9
10 sensores = pd.DataFrame({
11     "sensor1_temp": temperatura,
12     "sensor2_temp": temperatura + np.random.normal(0, 2, n),
13     "sensor3_presion": temperatura * 0.5 + np.random.normal(0, 5, n),
14     "sensor4_presion": temperatura * 0.45 + np.random.normal(0, 5, n),
15     "sensor5_vibracion": np.random.normal(20, 3, n),
16     "sensor6_vibracion": np.random.normal(21, 3, n),
17     "sensor7_flujo": np.random.normal(50, 8, n),
18     "sensor8_flujo": np.random.normal(49, 8, n),
19     "sensor9_CO2": temperatura * 0.3 + np.random.normal(0, 2, n),
20     "sensor10_CO2": temperatura * 0.35 + np.random.normal(0, 2, n),
21     "sensor11_humedad": np.random.normal(60, 7, n),
22     "sensor12_humedad": np.random.normal(62, 7, n)
23 })
24
25 print(sensores.head())
```

monitoreo_industrial.py X

No plots to show

Run plot-generating code in the Editor or IPython console to see your figures appear here. This pane only supports static images, so it can't display interactive plots like Bokeh, Plotly or Altair.

Help Variable Explorer Debugger Profiler Plots Files

Console 1/A X

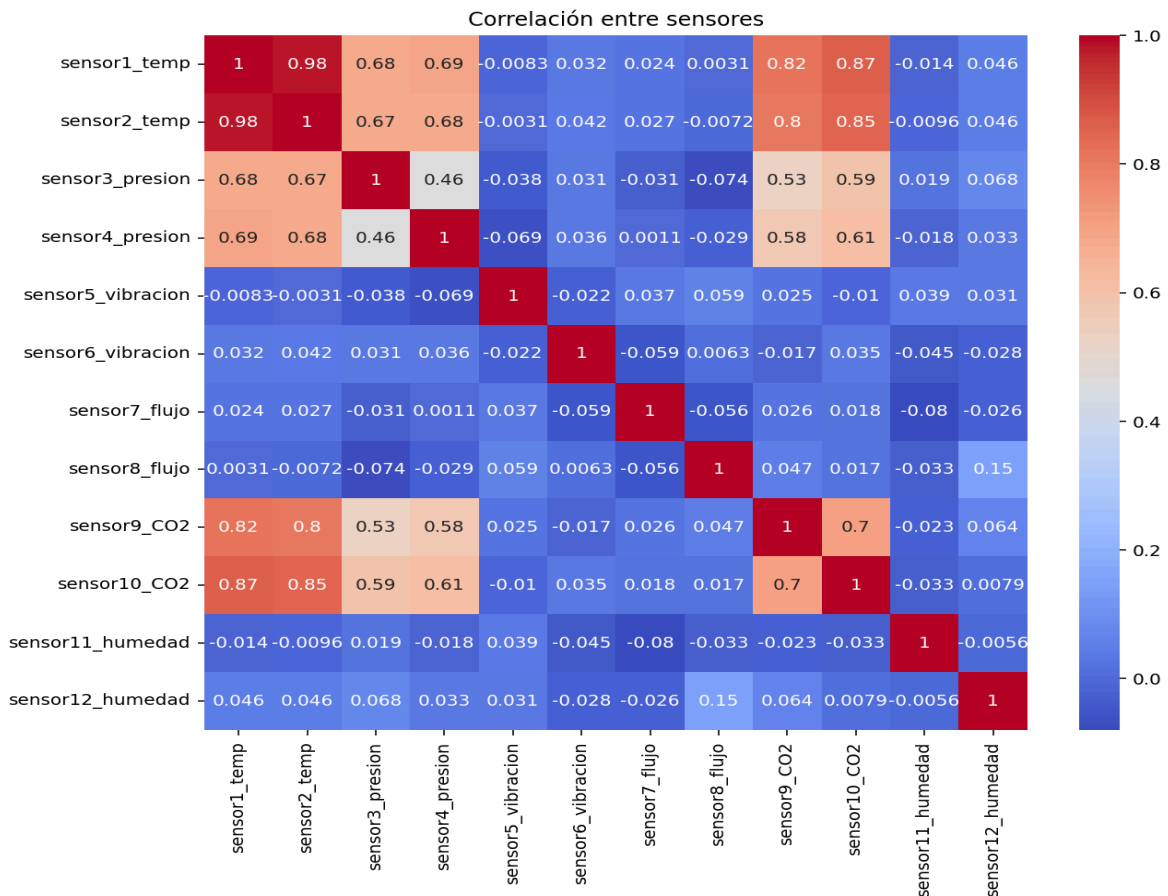
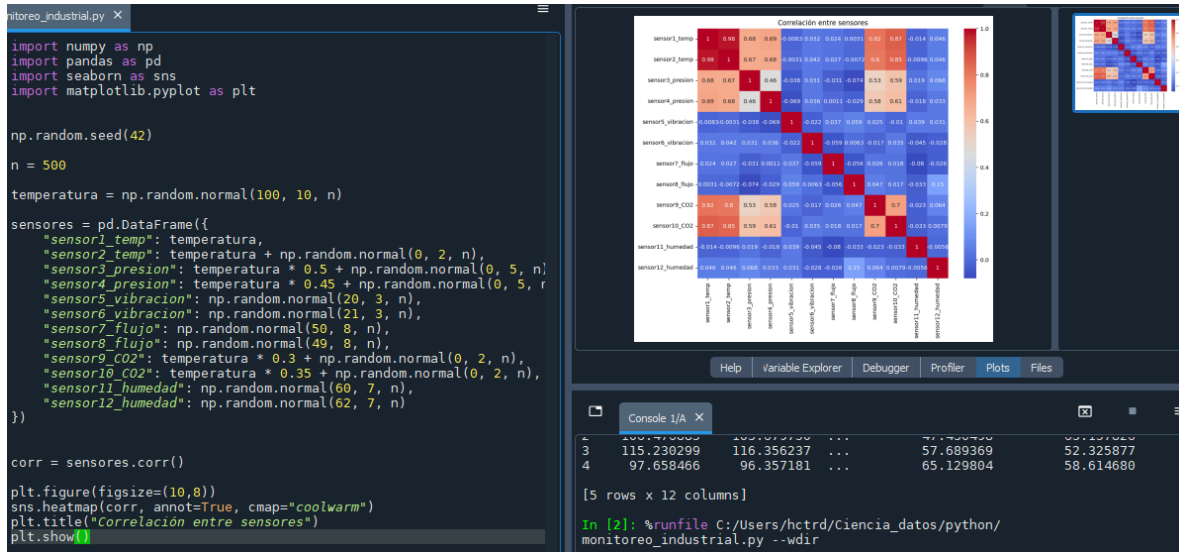
Restarting kernel...

```
In [1]: %runfile C:/Users/hctrd/Ciencia_datos/python/monitoreo_industrial.py -wdir
monitoreo_industrial.py --wdir
sensor1_temp sensor2_temp ... sensor11_humedad sensor12_humedad
0 104.967142 106.819497 ... 57.033682 66.921998
1 98.617357 102.436190 ... 56.826101 65.732451
2 106.476885 103.679750 ... 47.430498 63.157826
3 115.230299 116.356237 ... 57.689369 52.325877
4 97.658466 96.357181 ... 65.129804 58.614680

[5 rows x 12 columns]
```

Fase 1: Preparación

Graficamos la correlación entre los sensores para identificar aquellos que miden los mismos datos.



Fase 2: Ejecución de PCA

Para llevar a cabo el PCA primero debemos estandarizar los datos, después podemos agregar un ciclo for() para imprimir los resultados.

```
sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.title("Correlación entre sensores")

scaler = StandardScaler()
pca = PCA()

X_scaled = scaler.fit_transform(sensores)
componentes = pca.fit_transform(X_scaled)

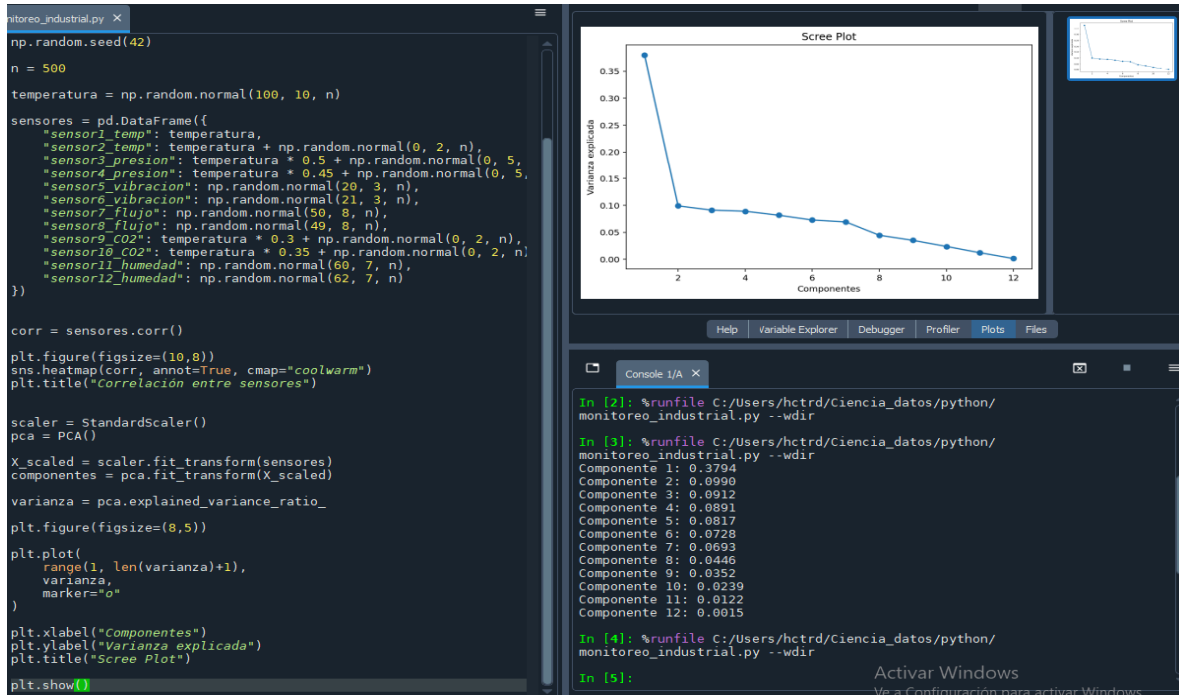
varianza = pca.explained_variance_ratio_

for i, v in enumerate(varianza):
    print(f"Componente {i+1}: {v:.4f}")
```

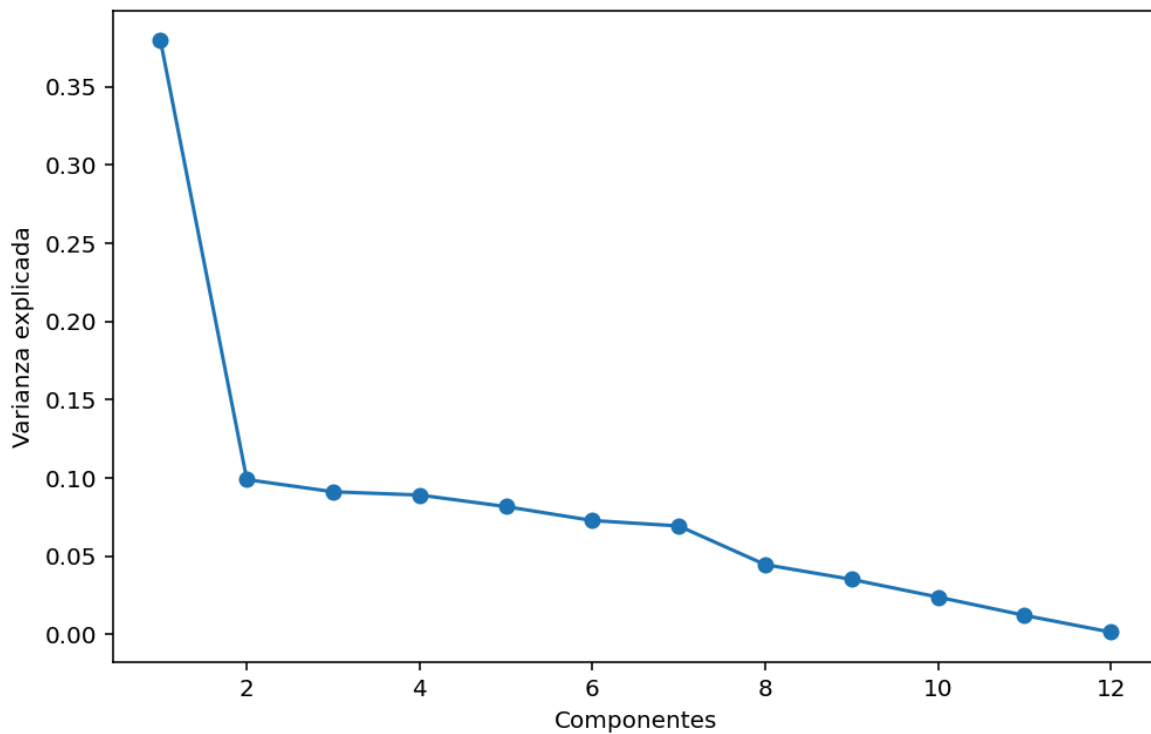
```
In [3]: %runfile C:/Users/hctrd/Ciencia_datos/python/
monitoreo_industrial.py --wdir
Componente 1: 0.3794
Componente 2: 0.0990
Componente 3: 0.0912
Componente 4: 0.0891
Componente 5: 0.0817
Componente 6: 0.0728
Componente 7: 0.0693
Componente 8: 0.0446
Componente 9: 0.0352
Componente 10: 0.0239
Componente 11: 0.0122
Componente 12: 0.0015
```

Fase 3: Selección de Componentes

Generamos el Scree Plot



Scree Plot



Calculamos la varianza acumulada y cuántos componentes son necesarios para alcanzar mínimo un 85% de la misma.

```
plt.ylabel('Varianza Explicada')
plt.title("Scree Plot")

plt.show()

varianza_acumulada = np.cumsum(varianza)

for i, v in enumerate(varianza_acumulada):
    print(f"{i+1} componentes -> {v:.4f}")

n_componentes = np.argmax(
    varianza_acumulada >= 0.85
) + 1

print(
    f"Se necesitan {n_componentes} componentes "
    "para explicar al menos el 85% de la varianza."
)
```

```
In [5]: %runfile C:/Users/hctrd/Ciencia_datos/python/
monitoreo_industrial.py --wdir
1 componentes -> 0.3794
2 componentes -> 0.4784
3 componentes -> 0.5695
4 componentes -> 0.6586
5 componentes -> 0.7403
6 componentes -> 0.8131
7 componentes -> 0.8825
8 componentes -> 0.9271
9 componentes -> 0.9623
10 componentes -> 0.9862
11 componentes -> 0.9985
12 componentes -> 1.0000
Se necesitan 7 componentes para explicar al menos el 85% de la varianza.

In [6]:
```

Calculamos los sensores originales que poseen más peso en el primer componente principal.

```
plt.show()

varianza_acumulada = np.cumsum(varianza)

n_componentes = np.argmax(
    varianza_acumulada >= 0.85
) + 1

loadings = pd.DataFrame(
    pca.components_.T,
    columns=[f"PC{i+1}" for i in range(len(sensores.columns))],
    index=sensores.columns
)

print(loadings["PC1"].sort_values(
    ascending=False
))
```

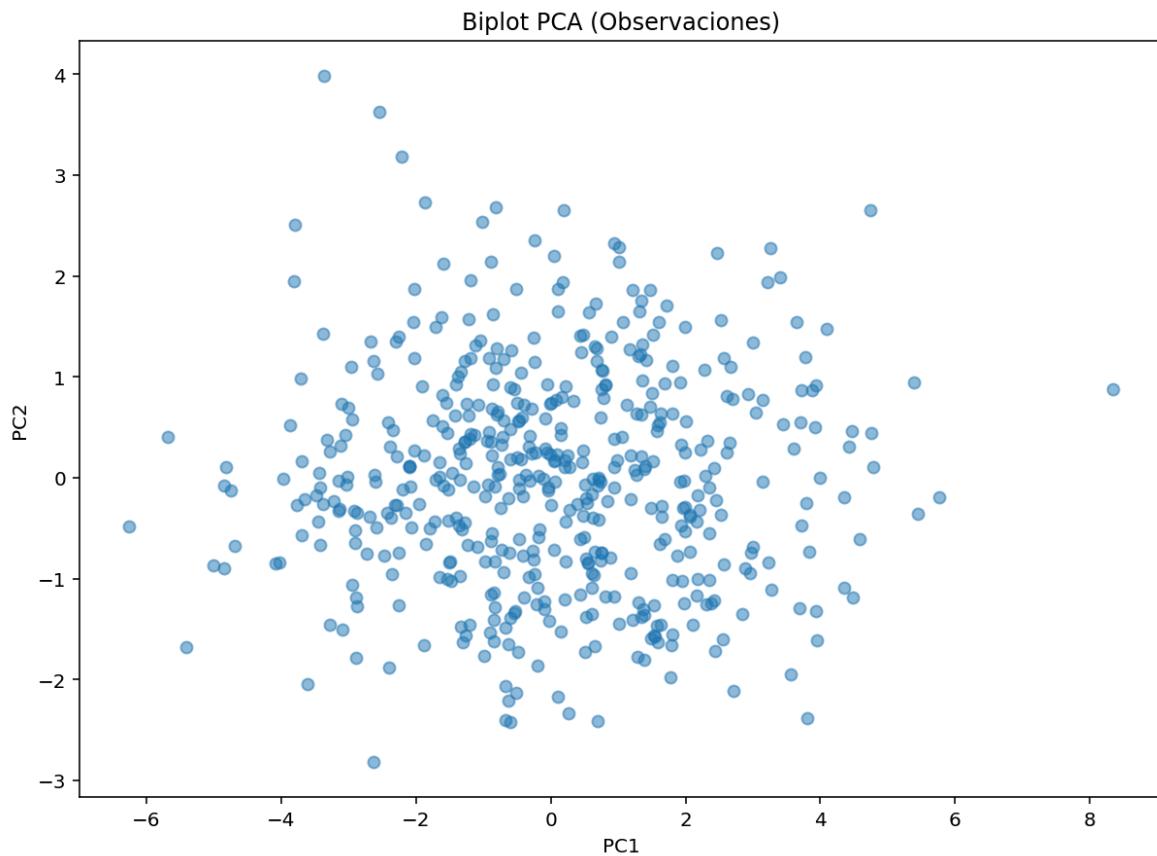
```
In [6]: %runfile C:/Users/hctrd/Ciencia_datos/python/
monitoreo_industrial.py --wdir
sensor1_temp      0.456381
sensor2_temp      0.451823
sensor10_CO2      0.419099
sensor9_CO2       0.400907
sensor4_presion   0.358312
sensor3_presion   0.348338
sensor12_humedad  0.029457
sensor6_vibracion  0.018095
sensor7_flujo     0.008314
sensor8_flujo     -0.002314
sensor11_humedad  -0.009894
sensor5_vibracion -0.010358
Name: PC1, dtype: float64

In [7]:
```

Fase 4: Visualización

Se creó el Biplot para visualizar los grupos de lotes de producción en el nuevo espacio de dimensiones reducidas.

```
72 plt.figure(figsize=(10,7))
73
74 plt.scatter(
75     componentes[:,0],
76     componentes[:,1],
77     alpha=0.5
78 )
79
80 plt.xlabel("PC1")
81 plt.ylabel("PC2")
82 plt.title("Biplot PCA (Observaciones)")
83
84 plt.show()
```



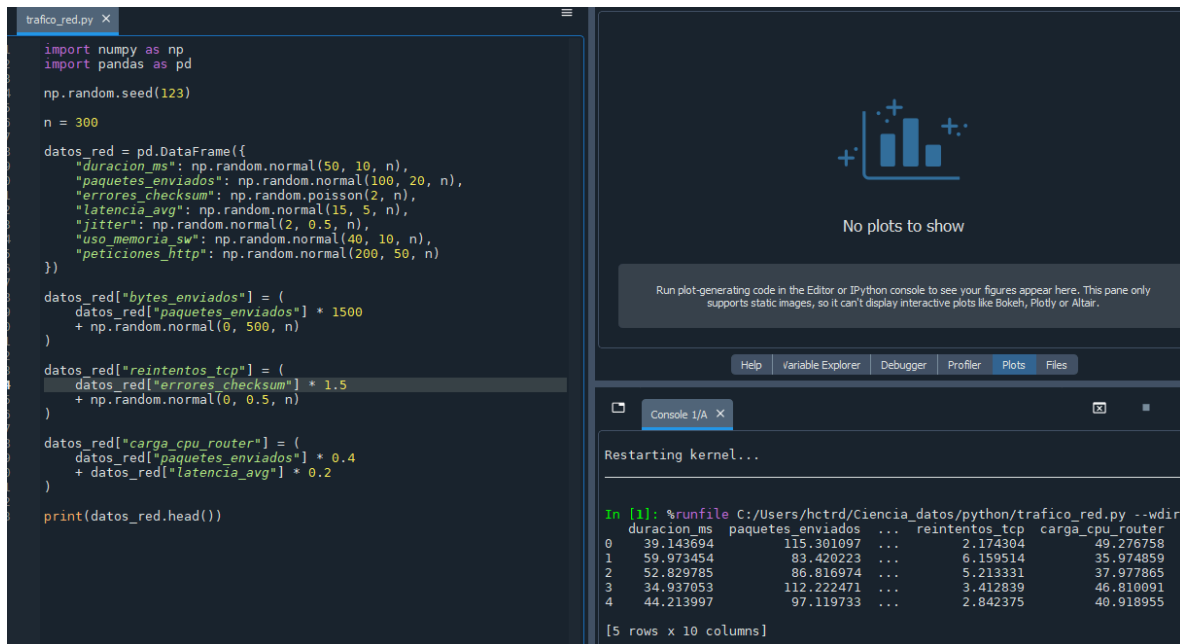
Conclusión

El análisis PCA permitió reducir las 12 variables originales a un número menor de componentes principales conservando más del 85% de la información del sistema. Los resultados mostraron una fuerte correlación entre algunos sensores de temperatura, presión y emisiones, indicando redundancia en las mediciones. El primer componente principal estuvo dominado por los sensores relacionados con temperatura y presión, sugiriendo que estos explican gran parte del comportamiento de la caldera. La reducción dimensional obtenida permitiría disminuir la carga de procesamiento y el consumo de ancho de banda del PLC sin perder información crítica para el monitoreo industrial.

Práctica 5: Reducción de Dimensiones en Tráfico de Red (Cyber-Systems)

Paso 1: Generación y Exploración de Datos

Generamos el DataFrame con la semilla y creamos las dependencias.



The screenshot shows a Jupyter Notebook interface with a code editor on the left and a console on the right. The code in the editor generates a DataFrame with 300 rows and 10 columns. The columns are: `duracion_ms`, `paquetes_enviados`, `errores_checksum`, `latencia_avg`, `jitter`, `uso_memoria_sw`, `peticiones_http`, `bytes_enviados`, `reintentos_tcp`, and `carga_cpu_router`. The console shows the output of the code, including the first 5 rows of the DataFrame.

```
import numpy as np
import pandas as pd

np.random.seed(123)

n = 300

datos_red = pd.DataFrame({
    "duracion_ms": np.random.normal(50, 10, n),
    "paquetes_enviados": np.random.normal(100, 20, n),
    "errores_checksum": np.random.poisson(2, n),
    "latencia_avg": np.random.normal(15, 5, n),
    "jitter": np.random.normal(2, 0.5, n),
    "uso_memoria_sw": np.random.normal(40, 10, n),
    "peticiones_http": np.random.normal(200, 50, n)
})

datos_red["bytes_enviados"] = (
    datos_red["paquetes_enviados"] * 1500
    + np.random.normal(0, 500, n)
)

datos_red["reintentos_tcp"] = (
    datos_red["errores_checksum"] * 1.5
    + np.random.normal(0, 0.5, n)
)

datos_red["carga_cpu_router"] = (
    datos_red["paquetes_enviados"] * 0.4
    + datos_red["latencia_avg"] * 0.2
)

print(datos_red.head())
```

Run plot-generating code in the Editor or IPython console to see your figures appear here. This pane only supports static images, so it can't display interactive plots like Bokeh, Plotly or Altair.

Help Variable Explorer Debugger Profiler Plots Files

Console 1/A X

Restarting kernel...

```
In [1]: %runfile C:/Users/hctrd/Ciencia_datos/python/trafico_red.py --wdir
duracion_ms  paquetes_enviados  ...  reintentos_tcp  carga_cpu_router
0    39.143694    115.301097  ...    2.174304    49.276758
1    59.973454    83.420223  ...    6.159514    35.974859
2    52.829785    86.816974  ...    5.213331    37.977865
3    34.937053    112.222471  ...    3.412839    46.810091
4    44.213997    97.119733  ...    2.842375    40.918955

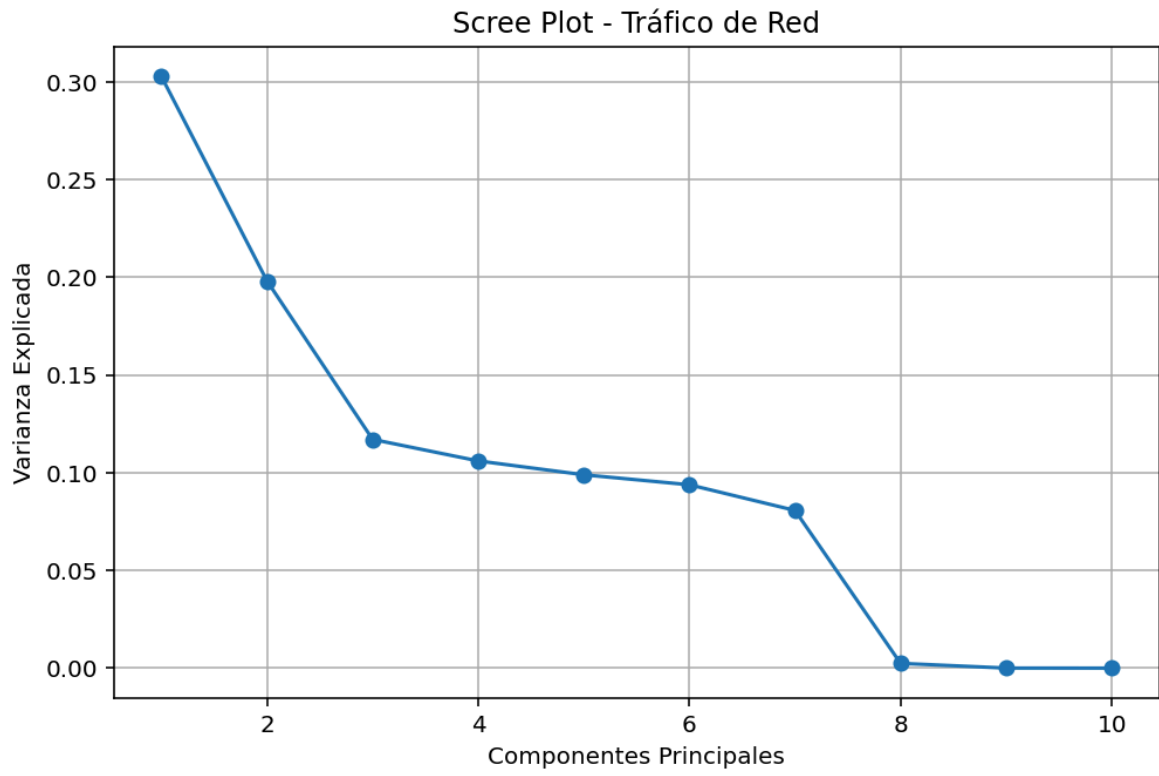
[5 rows x 10 columns]
```

Paso 2: Estandarización (Scaling)

Estandarizamos y aplicamos PCA.

```
trafico_red.py X
1  import numpy as np
2  import pandas as pd
3
4  from sklearn.preprocessing import StandardScaler
5  from sklearn.decomposition import PCA
6
7  np.random.seed(123)
8
9  n = 300
10
11  datos_red = pd.DataFrame({
12      "duracion_ms": np.random.normal(50, 10, n),
13      "paquetes_enviados": np.random.normal(100, 20, n),
14      "errores_checksum": np.random.poisson(2, n),
15      "latencia_avg": np.random.normal(15, 5, n),
16      "jitter": np.random.normal(2, 0.5, n),
17      "uso_memoria_sw": np.random.normal(40, 10, n),
18      "peticiones_http": np.random.normal(200, 50, n)
19  })
20
21  datos_red["bytes_enviados"] = (
22      datos_red["paquetes_enviados"] * 1500
23      + np.random.normal(0, 500, n)
24  )
25
26  datos_red["reintentos_tcp"] = (
27      datos_red["errores_checksum"] * 1.5
28      + np.random.normal(0, 0.5, n)
29  )
30
31  datos_red["carga_cpu_router"] = (
32      datos_red["paquetes_enviados"] * 0.4
33      + datos_red["latencia_avg"] * 0.2
34  )
35
36  scaler = StandardScaler()
37  pca_red = PCA()
38
39  X_scaled = scaler.fit_transform(datos_red)
40  componentes = pca_red.fit_transform(X_scaled)
```

Posteriormente generamos un ScreePlot.



Paso 3: Análisis de Varianza (Varianza Explicada)

Calculamos las varianzas.

```
datos_red["reintentos_tcp"] = (
    datos_red["errores_checksum"] * 1.5
    + np.random.normal(0, 0.5, n)
)

datos_red["carga_cpu_router"] = (
    datos_red["paquetes_enviados"] * 0.4
    + datos_red["latencia_avg"] * 0.2
)

scaler = StandardScaler()
pca_red = PCA()

X_scaled = scaler.fit_transform(datos_red)
componentes = pca_red.fit_transform(X_scaled)

varianza = pca_red.explained_variance_ratio_
varianza_acumulada = np.cumsum(varianza)

for i, v in enumerate(varianza):
    print(f"PC{i+1}: {v:.4f}")

for i, v in enumerate(varianza_acumulada):
    print(f"{i+1} componentes: {v:.4f}")
```

```
PC1: 0.3028
PC2: 0.1981
PC3: 0.1171
PC4: 0.1060
PC5: 0.0990
PC6: 0.0938
PC7: 0.0807
PC8: 0.0024
PC9: 0.0000
PC10: 0.0000
1 componentes: 0.3028
2 componentes: 0.5009
3 componentes: 0.6180
4 componentes: 0.7240
5 componentes: 0.8230
6 componentes: 0.9168
7 componentes: 0.9975
8 componentes: 1.0000
9 componentes: 1.0000
10 componentes: 1.0000

In [4]:
```

Paso 4: Interpretación de Cargas (Loadings)

Recuperamos los loadings.

```
scaler = StandardScaler()
pca_red = PCA()

X_scaled = scaler.fit_transform(datos_red)
componentes = pca_red.fit_transform(X_scaled)

varianza = pca_red.explained_variance_ratio_
varianza_acumulada = np.cumsum(varianza)

loadings = pd.DataFrame(
    pca_red.components_.T,
    columns=[f"PC{i+1}" for i in range(len(datos_red.columns))],
    index=datos_red.columns
)

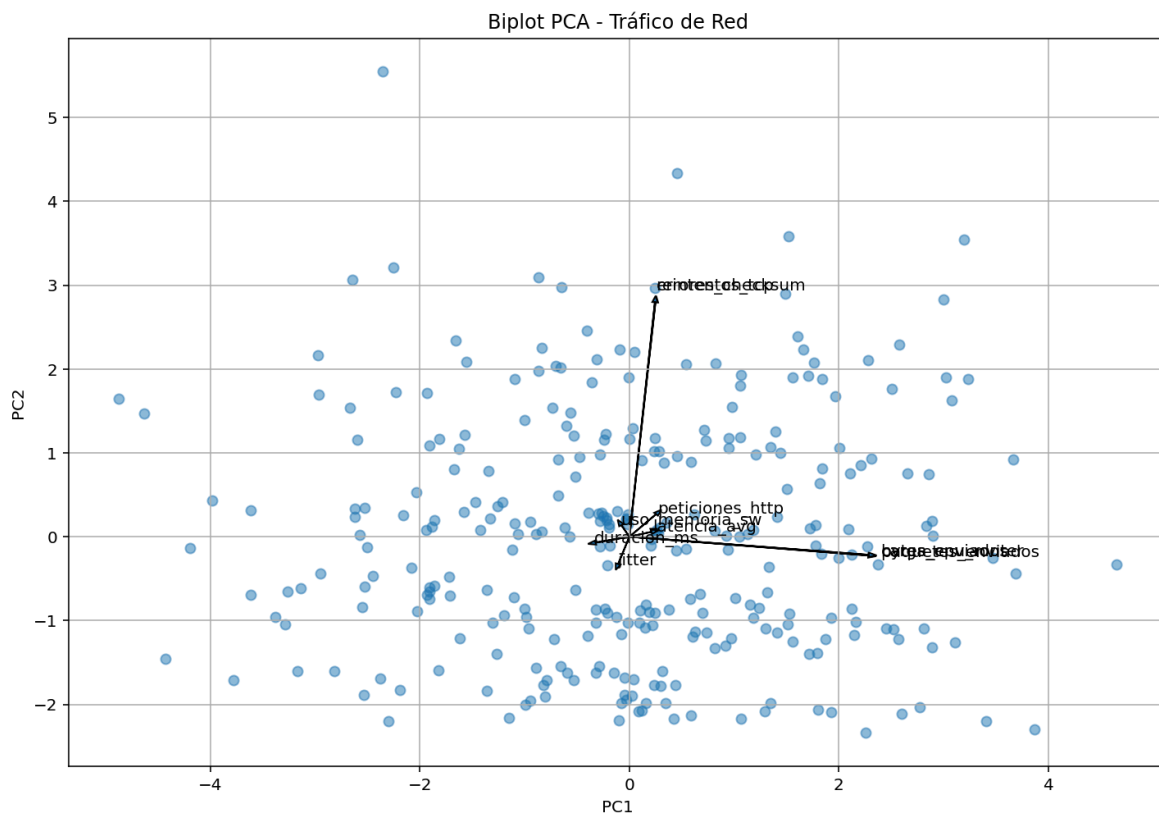
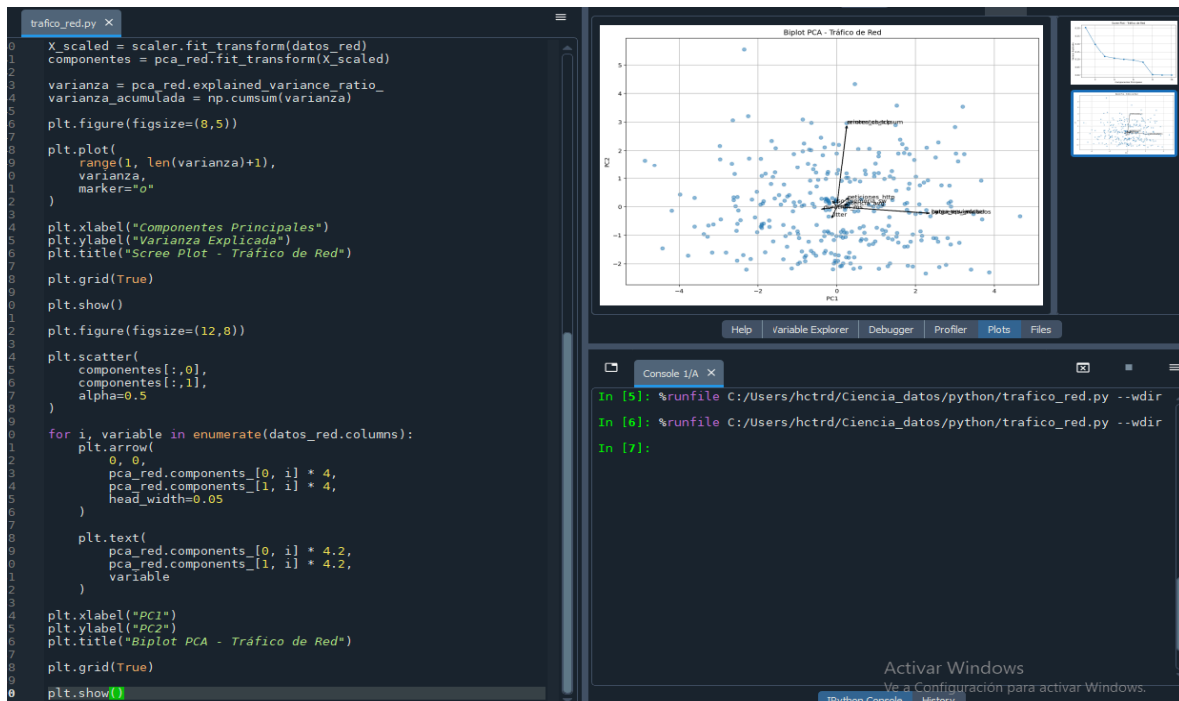
print(loadings[["PC1", "PC2"]].round(3))
```

```
In [4]: %runfile C:/Users/hctrd/C
PC1    PC2
duracion_ms      -0.081 -0.017
paquetes_enviados 0.571 -0.056
errores_checksum  0.062  0.699
latencia_avg      0.052  0.015
jitter           -0.027 -0.081
uso_memoria_sw    -0.019  0.034
peticiones_http   0.064  0.067
bytes_enviados     0.571 -0.055
reintentos_tcp     0.061  0.699
carga_cpu_router  0.572 -0.054

In [5]:
```

Paso 5: Visualización (Biplot)

Por último generamos un Biplot.



Conclusión

El análisis PCA permitió reducir las diez métricas originales de monitoreo de red a un conjunto más pequeño de componentes principales que conservan la mayor parte de la información. La matriz de correlación mostró una fuerte redundancia entre variables como paquetes_enviados y bytes_enviados, así como entre errores_checksum y reintentos_tcp.

La varianza explicada indicó que los primeros componentes concentran gran parte de la información del sistema, reduciendo significativamente la complejidad del análisis. Los loadings permitieron interpretar el primer componente como una medida del volumen de tráfico de red, mientras que el segundo componente representó principalmente la calidad y estabilidad de la conexión debido a la influencia de la latencia y el jitter.

El biplot permitió identificar relaciones entre variables y posibles observaciones atípicas que podrían corresponder a anomalías, fallas o intentos de ataque. En consecuencia, el uso de PCA representa una estrategia efectiva para disminuir la carga computacional del sistema de monitoreo sin perder información crítica para la detección temprana de incidentes.